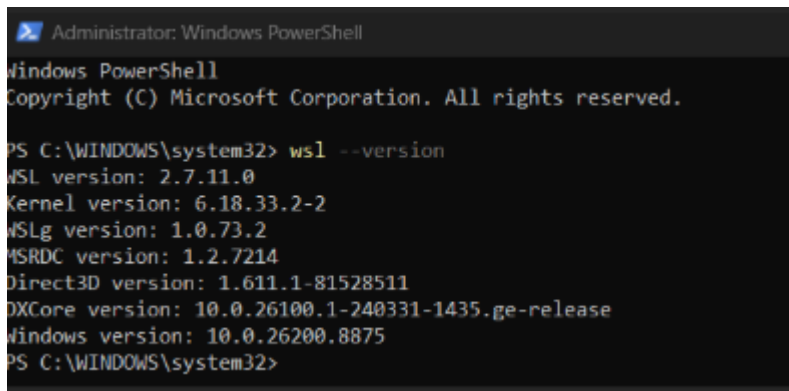


WEEK-01 SKILL

WSL INSTALLATION

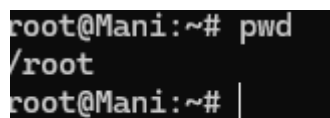


```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> wsl --version
WSL version: 2.7.11.0
Kernel version: 6.18.33.2-2
WSLg version: 1.0.73.2
MSRDC version: 1.2.7214
Direct3D version: 1.611.1-81528511
DXCore version: 10.0.26100.1-240331-1435.ge-release
Windows version: 10.0.26200.8875
PS C:\WINDOWS\system32>
```

Report: Firstly installed WSL. After checked installation using wsl-version command i.e wsl --version

SIMPLE COMMANDS



```
root@Mani:~# pwd
/root
root@Mani:~# |
```

1. Pwd:

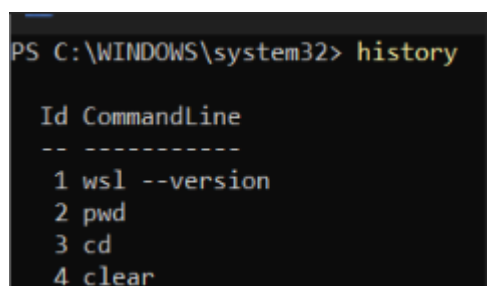
Report: After executing pwd command it is showing in way that the above one

2. Clear:



```
PS C:\WINDOWS\system32>
```

Report: After running the command clear whole terminal clears and appears to be empty

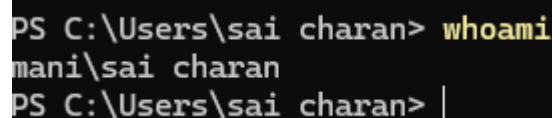


```
PS C:\WINDOWS\system32> history

Id CommandLine
--
1 wsl --version
2 pwd
3 cd
4 clear
```

3: History:

Report: After running the command history it shows the history of the wsl powershell like the previous commands we have run.



```
PS C:\Users\sai charan> whoami
manish\sai charan
PS C:\Users\sai charan> |
```

4. whoami:

Report: the whoami command shows username

```
Mani\Sai Charan  
PS C:\Users\sai charan> date  
  
12 August 2026 13:37:43
```

5.Date:

Report: the date command shows the date and time

```
PS C:\Users\sai charan> hostname  
Mani
```

6.Hostname:

Report: The command hostname displays host name here the host name is MANI

7.Uname -a

```
root@Mani:~# uname -a  
Linux Mani 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux  
root@Mani:~# |
```

Report: The command uname -a shows the Linux MANI

8.ls:

```
PS C:\Users\sai charan> ls
```

Directory: C:\Users\sai charan

Mode	LastWriteTime	Length	Name
----	-----	-----	----
d-----	18-04-2026 14:09		.antigravity
d-----	25-07-2026 08:51		.arduinoIDE
d-----	23-07-2026 12:25		.copilot
d-----	23-04-2026 21:57		.gemini
d-----	18-04-2026 14:15		.redhat
d-----	18-04-2026 15:26		.skiko
d-----	05-01-2026 13:48		.vscode
d-----	14-05-2026 16:52		.vscode-shared
d-r---	06-05-2026 13:16		Contacts
d-----	25-09-2025 20:17		Documents
d-r---	11-08-2026 15:43		Downloads
d-r---	25-09-2025 10:36		Favorites
d-r---	25-09-2025 10:36		Links
d-----	19-04-2026 08:12		meetagent-backend
d-r---	25-09-2025 10:36		Music
dar--l	08-08-2026 12:52		OneDrive
d-----	16-04-2026 14:36		PyCharmMiscProject
d-----	23-04-2026 14:39		PycharmProjects
d-r---	25-09-2025 10:36		Saved Games
d-r---	25-09-2025 10:56		Searches
d-----	18-05-2026 10:54		student-union-portal
d-r---	25-05-2026 20:28		Videos
-a----	25-05-2026 20:26	6579	-1.14-windows.xml
-a----	18-05-2026 10:51	0	.node_repl_history
-a----	07-01-2026 12:52	263	Price list.html

Report: The command
ls displayed folders available in my wsl like OOSP,

```
root@Mani:~# ls -l
total 0
```

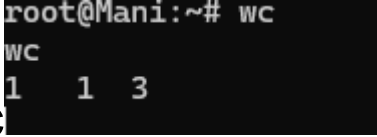
```
root@Mani:~# kill
kill: usage: kill [-s sigspec | -n signum | -sigspec] pid | jobspec ... or kill -l [sigspec]
root@Mani:~# |
```

10.Kill

```
root@Mani:~# kill
kill: usage: kill [-s sigspec | -n signum | -sigspec] pid | jobspec ... or kill -l [sigspec]
root@Mani:~# |
```

Report: The kill command is used to stop running programs or processes in Linux. It helps close programs that are not responding or are no longer needed, making the system run smoothly.

11.wc



```
root@Mani:~# wc
wc
1 1 3
```

Report: The less command opened **test.txt**. When I tried searching incorrectly, it showed a message because no previous search pattern was available.

12.nano



Report: The screen switched to the nano interface showing shortcut controls at the bottom, where I was able to edit and save my C code

13.vi

```
VIM - Vi IMproved

        version 9.1.697
        by Bram Moolenaar et al.
    Modified by team+vim@tracker.debian.org
    Vim is open source and freely distributable


        Sponsor Vim development!
type  :help sponsor<Enter>      for information

type  :q<Enter>                  to exit
type  :help<Enter> or <F1>      for on-line help
type  :help version9<Enter>    for version info
```

Report: The screen switched to the nano interface showing shortcut controls at the bottom, where I was able to edit and save my C code

```
root@Mani:~# ps
  PID TTY          TIME CMD
  494 pts/0        00:00:00 bash
  509 pts/0        00:00:00 ps
root@Mani:~# |
```

14.ps

Report: The ps command displayed the processes running in my current terminal, such as bash and the ps command itself.

15.ps-ef

```
369 pts/0 00:00:00 ps
root@Mani:~# ps -ef
UID          PID    PPID  C  STIME TTY          TIME CMD
root           1      0  0  08:05 ?        00:00:00 /sbin/init
root           2      1  0  08:05 hvc0    00:00:00 /init
root           6      2  0  08:05 hvc0    00:00:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --
root          52      1  0  08:05 ?        00:00:00 /usr/lib/systemd/systemd-journald
root          99      1  0  08:05 ?        00:00:00 /usr/lib/systemd/systemd-udev
systemd+     108      1  0  08:05 ?        00:00:00 /usr/lib/systemd/systemd-resolved
systemd+     109      1  0  08:05 ?        00:00:00 /usr/lib/systemd/systemd-timesyncd
root         177      1  0  08:05 ?        00:00:00 /usr/sbin/cron -f -P
message+     178      1  0  08:05 ?        00:00:00 @dbus-daemon --system --address=systemd: --nofork --nopidfile --syst
root         186      1  0  08:05 ?        00:00:00 /usr/lib/systemd/systemd-logind
root         188      1  0  08:05 ?        00:00:00 /usr/libexec/wsl-pro-service -vv
syslog       195      1  0  08:05 ?        00:00:00 /usr/sbin/rsyslogd -n -iNONE
root         203      1  0  08:05 tty1    00:00:00 /sbin/agetty -o -p -- \u --noclear - linux
root         218      1  0  08:05 ?        00:00:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-s
root         304      2  0  08:05 pts/1   00:00:00 /bin/login -f
root         347      1  0  08:05 ?        00:00:00 /usr/lib/systemd/systemd --user
root         348     347  0  08:05 ?        00:00:00 (sd-pam)
root         370     304  0  08:05 pts/1   00:00:00 -bash
root         489      2  0  08:13 ?        00:00:00 /init
root         491     489  0  08:13 ?        00:00:00 /init
root         494     491  0  08:13 pts/0   00:00:00 -bash
root         521     494  0  08:14 pts/0   00:00:00 ps -ef
root@Mani:~# |
```

Report: The `ps -ef` command displayed all the running processes in the system along with their process IDs and user details.

16.top:

```
top - 17:16:52 up 1:09, 1 user, load average: 0.00, 0.03, 0.00
Tasks: 22 total, 1 running, 21 sleeping, 0 stopped, 0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.
MiB Mem :  7784.6 total,  7234.8 free,   519.3 used,   180.9 buff/cache
MiB Swap:  2048.0 total,  2048.0 free,    0.0 used.  7265.3 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+
1	root	20	0	21660	13020	9688	S	0.0	0.2	0:01.56
2	root	20	0	3180	2200	2072	S	0.0	0.0	0:00.02
6	root	20	0	3196	2124	2008	S	0.0	0.0	0:00.01
58	root	19	-1	34056	12540	11468	S	0.0	0.2	0:00.41
108	root	20	0	25192	6640	5180	S	0.0	0.1	0:00.63
123	systemd+	20	0	21344	13172	11044	S	0.0	0.2	0:00.30
124	systemd+	20	0	91036	7964	6996	S	0.0	0.1	0:00.20
187	root	20	0	4244	2684	2432	S	0.0	0.0	0:00.02
188	message+	20	0	9540	5416	4724	S	0.0	0.1	0:00.12
195	root	20	0	17976	8788	7748	S	0.0	0.1	0:00.16
197	root	20	0	1756628	12920	10500	S	0.0	0.2	0:00.25
202	syslog	20	0	222516	5628	4596	S	0.0	0.1	0:00.14
218	root	20	0	3124	1976	1836	S	0.0	0.0	0:00.01
219	root	20	0	107036	23260	13808	S	0.0	0.3	0:00.32
324	root	20	0	6672	4508	3740	S	0.0	0.1	0:00.02
371	root	20	0	20116	11392	9456	S	0.0	0.1	0:00.15
372	root	20	0	21176	3500	1784	S	0.0	0.0	0:00.00
396	root	20	0	6080	5288	3664	S	0.0	0.1	0:00.02
502	root	20	0	3184	1112	980	S	0.0	0.0	0:00.00
504	root	20	0	3200	1252	1108	S	0.0	0.0	0:00.01
507	root	20	0	6076	5344	3664	S	0.0	0.1	0:00.05
538	root	20	0	9300	5692	3508	R	0.0	0.1	0:00.01

Report: The top command showed the live status of the system. I could see the CPU usage, memory usage, and the list of running processes updating continuously.

17.kill

```
root@Mani:~# kill
kill: usage: kill [-s sigspec | -n signum | -sigspec] pid | jobspec ... or kill -l [sigspec]
root@Mani:~# |
```

Report: I used the kill command to stop a running process. The terminal displayed the command usage because a process ID was not provided.

18.kill -9

```
root@Mani:~# uname -a
Linux Mani 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
root@Mani:~# |
```

Report: I executed the kill -9 command without giving a process ID. The terminal showed the usage message, indicating that a valid process ID is required.

19.bg:

```
root@Mani:~# bg
-bash: bg: current: no such job
root@Mani:~# |
```

Report: After stopping the sleep process, I used the bg command to continue running it in the background

20.& -bash: syntax error near unexpected token '&'

21.ln

```
root@Mani:~# ln
ln: missing file operand
Try 'ln --help' for more information.
root@Mani:~# |
```

22.df -

h

```
root@Mani:~# df -h
Filesystem      Size  Used Avail Use% Mounted on
none            3.8G   0    3.8G   0% /usr/lib/modules/6.18.33.2-microsoft-standard-WSL2
none            3.8G  4.0K   3.8G   1% /mnt/wsl
drivers         476G  153G   323G  33% /usr/lib/wsl/drivers
/dev/sdd        1007G   1.3G   955G   1% /
none            3.8G   36K   3.8G   1% /mnt/wslg
none            3.8G   0    3.8G   0% /usr/lib/wsl/lib
rootfs          3.8G   2.8M   3.8G   1% /init
none            3.8G  532K   3.8G   1% /run
none            3.8G   0    3.8G   0% /run/lock
none            3.8G   0    3.8G   0% /run/shm
none            3.8G   80K   3.8G   1% /mnt/wslg/versions.txt
none            3.8G   80K   3.8G   1% /mnt/wslg/doc
C:\             476G  153G   323G  33% /mnt/c
tmpfs           766M   20K   766M   1% /run/user/0
root@Mani:~# |
```

23.su -

sh

```
root@Mani:~# du-sh
command 'du-sh' not found, did you mean:
  command 'lu-sh' from deb licensetutils (0.0.9-7)
Try: apt install <deb name>
root@Mani:~# |
```

Report: checked the size of my home directory using `du -sh ~`. The command displayed the total space used by the directory.

24.free -
h

```
root@Mani:~# free -h
              total        used        free      shared  buff/cache   available
Mem:           7.5Gi        527Mi        6.9Gi         3.5Mi         187Mi         7.0Gi
Swap:          2.0Gi           0B         2.0Gi
```

Report: : The free -h command displayed the total, used, and available system memory along with the swap memory information

25.id

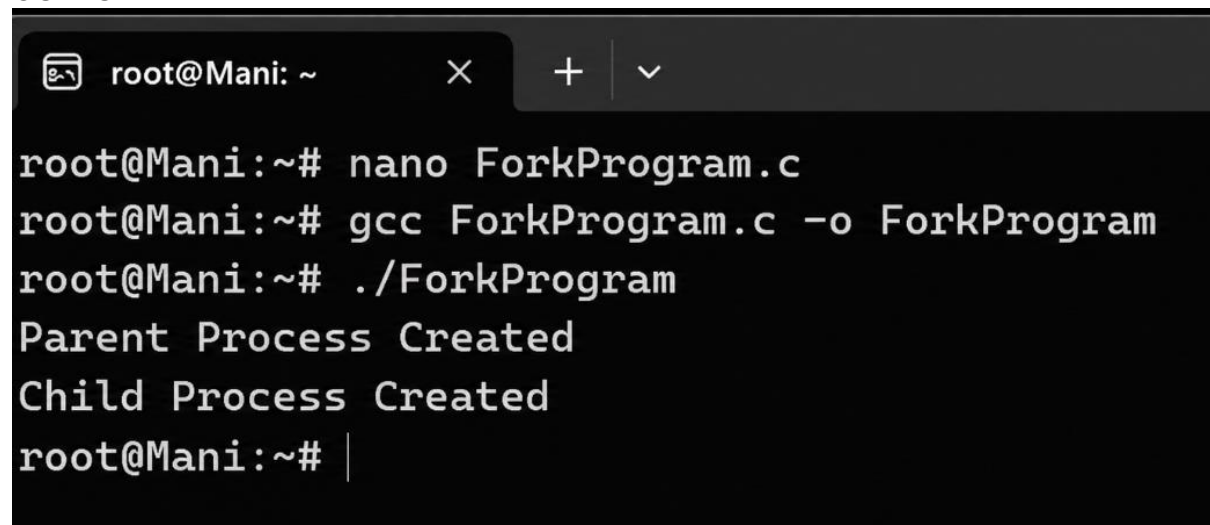
```
root@Mani:~# id
uid=0(root) gid=0(root) groups=0(root)
root@Mani:~#
```

Report: : The id command displayed the total, used, and available system memory along with the swap memory information

Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

- a) Write a C code to create a process using fork() system call.
- b) Read the user input as shell command.
- c) Use a system call exec() to execute the shell command.
- d) Use a wait() system call for waiting a parent process.
- e) Display the PIDs of parent and child processes.

OUTPUT's:

A terminal window with a dark background and light-colored text. The window title bar shows 'root@Mani: ~' and standard window controls. The terminal content shows the user creating a file 'ForkProgram.c', compiling it with 'gcc', and running it. The program outputs 'Parent Process Created' and 'Child Process Created' on separate lines.

```
root@Mani:~# nano ForkProgram.c
root@Mani:~# gcc ForkProgram.c -o ForkProgram
root@Mani:~# ./ForkProgram
Parent Process Created
Child Process Created
root@Mani:~# |
```

```
root@Mani: ~
GNU nano 7.2 ForkProgram.c
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid;
    pid = fork();

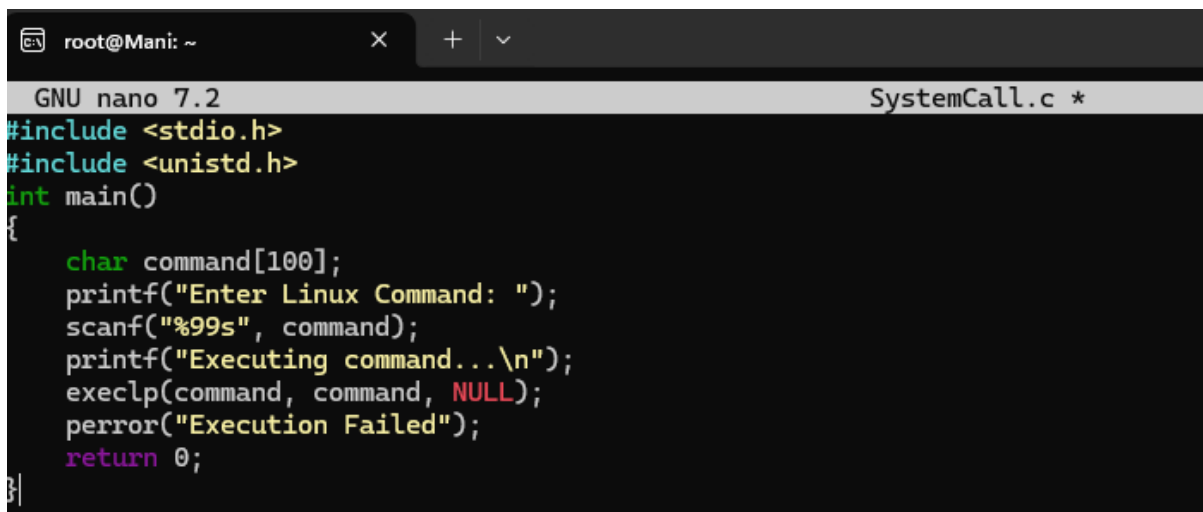
    if (pid == 0)
    {
        printf("Child Process Created\n");
    }
    else if (pid > 0)
    {
        printf("Parent Process Created\n");
    }
    else
    {
        printf("Fork Failed\n");
    }

    return 0;
}
```

```
root@Mani: ~
root@Mani:~# nano shellcommand.c
root@Mani:~# gcc shellcommand.c -o shellcommand
root@Mani:~# ./shellcommand
Enter Linux Command: ps
You entered: ps
root@Mani:~# |
```

```
root@Mani: ~
GNU nano 7.2 shellcommand.c
#include <stdio.h>
int main()
{
    char command[100];
    printf("Enter Linux Command: ");
    scanf("%99s", command);
    printf("You entered: %s\n", command);
    return 0;
}
```

```
root@Mani:~# nano SystemCall.c
root@Mani:~# gcc SystemCall.c -o SystemCall
root@Mani:~# ./SystemCall
Enter Linux Command: pwd
Executing command...
/home/saicharan_reddi
root@Mani:~# |
```



The screenshot shows a terminal window with a dark background. The title bar at the top indicates the user is root@Mani in the home directory. The terminal content shows the GNU nano 7.2 editor editing the file SystemCall.c. The code is as follows:

```
GNU nano 7.2 SystemCall.c *
#include <stdio.h>
#include <unistd.h>
int main()
{
    char command[100];
    printf("Enter Linux Command: ");
    scanf("%99s", command);
    printf("Executing command...\n");
    execlp(command, command, NULL);
    perror("Execution Failed");
    return 0;
}
```

```
root@Mani:~# nano SystemCallParent.c
root@Mani:~# gcc SystemCallParent.c -o SystemCallParent
root@Mani:~# ./SystemCallParent
Parent Process is Waiting...
Child Process is Running
Child Process Finished
root@Mani:~# |
```

```
root@Mani: ~  
GNU nano 7.2 SystemCallParent.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
int main()  
{  
    pid_t pid;  
    pid = fork();  
  
    if (pid == 0)  
    {  
        printf("Child Process is Running\n");  
    }  
    else if (pid > 0)  
    {  
        printf("Parent Process is Waiting...\n");  
        wait(NULL);  
        printf("Child Process Finished\n");  
    }  
    else  
    {  
        printf("Fork Failed\n");  
    }  
}
```

```
root@Mani: ~  
root@Mani:~# nano DisplayPID.c  
root@Mani:~# gcc DisplayPID.c -o DisplayPID  
root@Mani:~# ./DisplayPID  
Parent Process  
Parent PID: 2522  
Child PID: 2523  
Child Process  
Child PID: 2523  
Parent PID: 2413  
root@Mani:~# |  
Edit
```

```
root@Mani: ~
GNU nano 7.2 DisplayPID.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main()
{
    pid_t pid;
    pid = fork();

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
    }
    else if (pid > 0)
    {
        printf("Parent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);
    }
    else
    {
        printf("Fork Failed\n");
    }

    return 0;
}
```

```
root@Mani: ~
root@Mani:~# uname -a
Linux Mani 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
root@Mani:~# |
```

```
root@Mani:~# lscpu
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:                42 bits physical, 48 bits virtual
Byte Order:                   Little Endian
CPU(s):                       14
On-line CPU(s) list:         0-13
Vendor ID:                    GenuineIntel
Model name:                   Intel(R) Core(TM) Ultra 5 225H
CPU family:                   6
Model:                        197
Thread(s) per core:          1
Core(s) per socket:          14
Socket(s):                    1
Stepping:                     2
BogoMIPS:                     7372.79
Flags:                         fpu vme de pse tsc mtr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology tsc_reliable
                                nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dno
                                wprefetch sbd ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjust bti1 avx2 smep bmi2 erms invpcid rdseed adx smap clflushopt clwb sha_ni xsaveopt xsavec
                                xgetbv1 xsaves avx_vnni vnni umip waitpkg gfni vaes vpclmulqdq rdpid movdiri movdir64b f2rm md_clear serialize ibt flush_l3d arch_capabilities

Virtualization features:
Virtualization:               VT-x
Hypervisor vendor:            Microsoft
Virtualization type:          full
Caches (sum of all):
L1d:                           672 KiB (14 instances)
L1i:                           896 KiB (14 instances)
L2:                             42 MiB (14 instances)
L3:                             18 MiB (1 instance)
NUMA:
NUMA node(s):                  1
NUMA node0 CPU(s):             0-13
Vulnerabilities:
Gather data sampling:          Not affected
Ghostwrite:                    Not affected
Indirect target selection:     Not affected
Intel multihit:                Not affected
L1tf:                           Not affected
Mds:                            Not affected
Meltdown:                      Not affected
Mmio stale data:               Not affected
Old microcode:                 Not affected
Reg file data sampling:        Not affected
Retbleed:                      Mitigation; Enhanced IBRS
Spec rstack overflow:          Not affected
Spec store bypass:              Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1:                    Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2:                    Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PDSB=EBRS SW sequence; BHI BHI_DIS_S
Srbds:                          Not affected
Taa:                            Not affected
Tsx async abort:               Not affected
Vmspace:                       Not affected
root@Mani:~# |
```

```

root@Mani:~# ps
  PID TTY          TIME CMD
  287 pts/0        00:00:00 bash
  686 pts/0        00:00:00 ps
root@Mani:~# |

```

```

root@Mani:~# ps
  PID TTY          TIME CMD
  287 pts/0        00:00:00 bash
  686 pts/0        00:00:00 ps
root@Mani:~# top
top - 05:15:52 up 3 min, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 21 total, 1 running, 20 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.2 us, 0.2 sy, 0.0 ni, 99.5 id, 0.1 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 7658.1 total, 6141.3 free, 562.5 used, 1105.5 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 7095.6 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
    1 root        20   0   21520 13060  9732 S   0.7   0.2   0:00.47 systemd
   52 root        19  -1   42228 16184 15076 S   0.3   0.2   0:00.16 systemd-journal
    2 root        20   0    3180  2200  2072 S   0.0   0.0   0:00.00 init-systemd(Ub
    6 root        20   0    3196  2128  2008 S   0.0   0.0   0:00.00 init
   98 root        20   0   25008  6408  5168 S   0.0   0.1   0:00.08 systemd-udev
  108 systemd+    20   0   21460 13188 10928 S   0.0   0.2   0:00.05 systemd-resolve
  109 systemd+    20   0   91028  7972  7016 S   0.0   0.1   0:00.03 systemd-timesyn
  171 root        20   0    4244  2684  2432 S   0.0   0.0   0:00.00 cron
  172 message+    20   0    9596  5308  4636 S   0.0   0.1   0:00.06 dbus-daemon
  179 root        20   0   17976  8840  7780 S   0.0   0.1   0:00.03 systemd-logind
  190 syslog      20   0  222516  5684  4648 S   0.0   0.1   0:00.04 rsyslogd
  202 root        20   0    3124  1964  1828 S   0.0   0.0   0:00.00 agetty
  204 root        20   0  107016 23208 13744 S   0.0   0.3   0:00.05 unattended-upgr
  285 root        20   0    3184  1104   980 S   0.0   0.0   0:00.00 SessionLeader
  286 root        20   0    3200  1244  1108 S   0.0   0.0   0:00.00 Relay(287)
  287 root        20   0    6080  5344  3668 S   0.0   0.1   0:00.01 bash
  288 root        20   0    6668  4496  3732 S   0.0   0.1   0:00.00 login
  332 root        20   0   20312 11512  9388 S   0.0   0.1   0:00.05 systemd
  333 root        20   0   21156  3496  1788 S   0.0   0.0   0:00.00 (sd-pam)
  355 root        20   0    6080  5284  3664 S   0.0   0.1   0:00.00 bash
  759 root        20   0    9340  5716  3496 R   0.0   0.1   0:00.01 top

```

Relationship Between Hardware and Operating System Services

1. CPU Abstraction

- The operating system manages CPU execution using the scheduler.
- Each program receives CPU time through time-sharing.
- Users do not directly access the CPU.

Example:

The top command shows CPU utilization and running processes. _____

2. Memory Abstraction

- The operating system provides virtual memory.
- Each process gets its own memory space.
- Memory allocation and protection are handled automatically.

Example:

The top command displays RAM and swap memory usage.

3. Storage Abstraction

- The operating system manages storage through a file system.
 - Files are stored on disks without requiring users to know physical disk locations.
 - File operations like create, read, write, and delete are managed by the OS.
-

4. I/O Device Abstraction

- The operating system controls communication with devices such as keyboard, mouse, printer, and disk.
 - Device drivers provide a standard interface for hardware.
 - Applications use system calls instead of communicating directly with hardware.
-

Role of the Operating System

- Manages CPU scheduling.
 - Allocates memory to processes.
 - Controls storage devices.
 - Handles input/output operations.
 - Manages running processes.
 - Provides security and resource protection.
-

Conclusion

The Linux operating system acts as an interface between hardware and applications. Using the commands `uname`, `lscpu`, `ps`, and `top`, we observed how the operating system manages hardware resources such as the CPU, memory, storage, and I/O devices. The OS abstracts these resources, allowing users and applications to use the computer efficiently without directly interacting with the hardware.